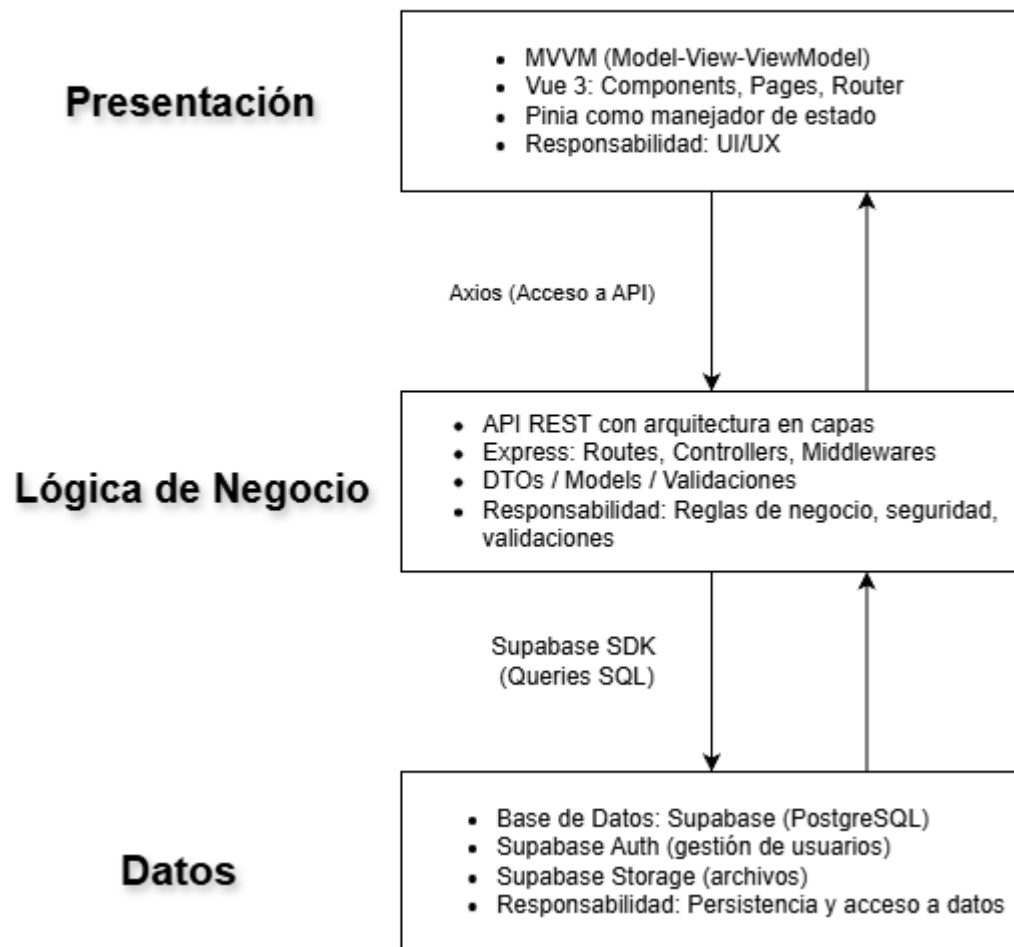


Arquitectura de Software

Arquitectura por Capas



Elección de la arquitectura:

La elección de esta arquitectura se basó principalmente en la separación de módulos, mejorando la mantenibilidad, escalabilidad, trazabilidad y la seguridad. Las capas de nuestra arquitectura son la de presentación, Lógica de negocio y la de datos.

Esta arquitectura nos permite mayor mantenibilidad y escalabilidad, aislando y estructurando en módulos, permitiendo una mejor organización visual del código.

Capa de Presentación

Es la encargada de la interfaz y experiencia de usuario. En esta se construye la interfaz completa de nuestra plataforma. En esta capa no se define lógica de negocio y solo se consumen servicios desde la capa de lógica de negocios.

Patrón de diseño:

Patrón de diseño MVVM (Model-View-ViewModel), separa la vista de la lógica de presentación, logrando componentes mas mantenibles. Se utiliza debido a que el Framework Vue, utilizado para el desarrollo, Trabaja con componentes que siguen este patrón de diseño.

Framework utilizado

Vue es un Framework de javascript destacado por tener una curva de aprendizaje suave. Esto nos permite aprender de forma mas rápida y sencilla esta tecnología.

Vue utiliza los siguientes elementos:

- Componentes: componentes reutilizables en las vistas.
- Pages: Vistas completas que se muestran en la interfaz de usuario.
- Router: Acceso a través de rutas a las vistas.

Centralización de estado

Se utilizo además un gestor de estados Pinia, que nos permite centralizar los estados de las variables del front end. Así es mas fácil acceder a los estados y el código es mas mantenible y estructurado.

Acceso a DB

Axios es la librería utilizada para realizar consultar por medio de llamadas con protocolo HTTP/HTTPS hacia la API que nos permite disponer de los servicios desde la capa de lógica de negocios.

Capa de Lógica de Negocios

Modulo que se encarga de procesar la lógica del sistema, la seguridad, validaciones y reglas de negocio.

Actúa como intermediarios entre la capa de presentación y la de datos, permitiendo mayor seguridad, restringiendo el acceso directo a los datos. Además, se realizan las validaciones y se aplican las reglas de negocio que nos permiten garantizar la integridad y consistencia de la información antes de que sea procesada o almacenada.

Patrón de diseño (continuación)

La arquitectura interna del backend sigue el enfoque por capas, donde se separan rutas, controladores, modelos, DTOs y middlewares. Esto permite que cada parte del sistema tenga una función definida, facilitando la mantenibilidad, la escalabilidad y la trazabilidad del código.

Además, se estructura bajo principios como responsabilidad única, lo que evita duplicidad de lógica y mejora la claridad en la ejecución de las reglas de negocio.

Tecnologías utilizadas

Para la implementación de la capa de lógica de negocios se utiliza:

- **Express.js:** Framework de Node.js que nos permite construir la API REST mediante la definición de rutas, controladores y middlewares.
- **Routes:** Definen los puntos de acceso a la API, especificando las operaciones disponibles para el frontend.
- **Controllers:** Contienen la lógica de procesamiento que maneja cada solicitud, organizando las acciones del sistema.
- **Middlewares:** Permiten ejecutar funciones adicionales entre la solicitud y la respuesta, como validaciones, seguridad, manejo de errores o sanitización de datos.
- **DTOs / Models / Validaciones:**
 - Los DTOs permiten normalizar la estructura de los datos que recibe o entrega la API.
 - Los modelos representan estructuras relacionadas al almacenamiento y manejo de información.
 - Las validaciones garantizan consistencia y evitan datos corruptos o malformados.

Acceso a Datos desde el Backend

Para interactuar con la base de datos y los servicios externos se utiliza el **SDK de Supabase**, que permite realizar consultas SQL, gestionar usuarios, interactuar con el almacenamiento y acceder a funcionalidades de autenticación.

Este enfoque permite que el backend controle completamente las operaciones que van a la base de datos, asegurando que solo datos válidos y limpios sean enviados o modificados.

Capa de Datos

La capa de datos es la encargada de administrar la persistencia, gestión y seguridad de toda la información del sistema. Esta capa funciona como el núcleo donde se almacena y organiza la información crítica y, gracias a los servicios de Supabase, ofrece un conjunto de herramientas integradas.

Servicios utilizados

Dentro de esta capa, se consideran los siguientes servicios fundamentales:

- **Base de Datos Supabase (PostgreSQL)**

Es la base central del sistema, donde se guardan registros estructurados. PostgreSQL es

elegido debido a su robustez, estabilidad y capacidad para manejar consultas complejas.

- **Supabase Auth**

Servicio encargado del manejo de usuarios: registro, login, recuperación de contraseñas y control de permisos. Este módulo garantiza la seguridad del sistema evitando accesos no autorizados.

- **Supabase Storage**

Gestión de archivos como imágenes, documentos u otros recursos estáticos necesarios para la plataforma. Permite almacenar y acceder a estos elementos de forma segura.

Responsabilidades

La capa de datos cumple con diversas responsabilidades críticas:

- Gestionar la persistencia de la información.
- Mantener la integridad y coherencia de la base de datos.
- Controlar los accesos mediante autenticación y autorizaciones.
- Asegurar que los datos estén disponibles para el backend de forma estructurada y segura.
- Almacenar y servir archivos o recursos estáticos cuando la plataforma lo requiera.

Comunicación entre capas

El flujo de la arquitectura mantiene un orden específico que refuerza la seguridad y la separación de responsabilidades:

1. **Capa de Presentación → Capa de Lógica de Negocios**

El frontend realiza solicitudes HTTP/HTTPS a la API utilizando Axios. Estas solicitudes representan peticiones de datos o envío de información para ser procesada.

2. **Capa de Lógica de Negocios → Capa de Datos**

El backend procesa la solicitud, valida la información, aplica reglas de negocio y, si es necesario, interactúa con la base de datos mediante el SDK de Supabase.

3. **Capa de Datos → Capa de Lógica de Negocios**

La base de datos retorna una respuesta al backend, ya sea datos solicitados, confirmación de operaciones o errores estructurados.

4. **Capa de Lógica de Negocios → Capa de Presentación**

El backend envía al frontend la información procesada, la cual será visualizada o utilizada por el usuario.